

SVX Library Class Catalog

SystemVerilog Extension Library — Core Library (**src/**)

Version 2.1.0 beta

Mark Glasser

September 2026

Contents

1	Types	3
1.1	svx_types — Standard Typedefs	3
1.2	type_handle	3
1.3	type_match	4
1.4	typeid	5
2	Traits	7
2.1	Traits Classes	7
3	Containers	10
3.1	object — Base Object	10
3.2	container — Container Base Classes	10
3.3	vector	11
3.4	map	12
3.5	queue	13
3.6	stack	14
3.7	deque	14
3.8	circ_buf — Circular Buffer	15
3.9	pair / triple / quadruple	15
3.10	sorter	16
4	Iterators	18
4.1	Iterator Interfaces	18
4.2	List Iterators	19
4.3	Map Iterators	21
4.4	Permutation Iterators	22
4.5	range	23
4.6	map_range	24
5	Algorithms	26
5.1	algo — Core Algorithms	26
5.2	algo2 — Dual-Iterator Algorithms	27
5.3	accum — Accumulation Algorithms	27
5.4	fcn — Function Objects	28
5.5	predicate — Predicate Classes	29
6	Linked Structures	31
6.1	node	31
6.2	tree	31
6.3	tree_iterator	32

7	Set	34
7.1	set	34

Types

1.1 svx_types — Standard Typedefs

File: src/types/svx_types.svh

Standardized C-style integer typedefs, size and index types, process identifiers, and package-level Boolean constants. Provides a portable numeric vocabulary across the library.

Typedef	Base Type	Description
int8_t	byte	Signed 8-bit integer
uint8_t	byte unsigned	Unsigned 8-bit integer
int16_t	shortint	Signed 16-bit integer
uint16_t	shortint unsigned	Unsigned 16-bit integer
int32_t	int	Signed 32-bit integer
uint32_t	int unsigned	Unsigned 32-bit integer
int64_t	longint	Signed 64-bit integer
uint64_t	longint unsigned	Unsigned 64-bit integer
int128_t	bit [127:0]	Signed 128-bit integer (two-state)
uint128_t	bit unsigned [127:0]	Unsigned 128-bit integer (two-state)
size_t	uint64_t	Container/array size
index_t	uint64_t	Unsigned container index
signed_index_t	int64_t	Signed container index (for skip/offset)
pid_t	int unsigned	Process identifier

1.2 type_handle

File: src/types/type_handle.svh

Singleton type-handle mechanism. `type_handle#(T)` is a parameterized class that returns a unique singleton object for each distinct type T, enabling runtime type identity comparison. `type_container#(T)` wraps any value with its type handle for polymorphic storage.

`type_handle_base`

virtual class `extends` object

Non-parameterized base for all type handles.

Method	Return	Args	Description
<code>get_type_handle</code>	<code>type_handle_base</code>	—	Return the singleton type handle for this object (pure virtual).

`type_handle#(T)`

class **extends** `type_handle_base`

Parameterized singleton. One instance exists per distinct type T.

Method	Return	Args	Description
<code>get_type</code>	<code>type_handle#()</code>	—	Return (or lazily create) the singleton for type T. Static.
<code>get_type_handle</code>	<code>type_handle_base</code>	—	Return this singleton cast to base type.

`type_container_base`

class

Non-parameterized base for typed containers; supports polymorphic storage with `convert2string`.

Method	Return	Args	Description
<code>convert2string</code>	<code>string</code>	—	Return a human-readable representation (default: empty string).
<code>get_type_handle</code>	<code>type_handle_base</code>	—	Return the type handle for the contained type (default: null).

`type_container#(T)`

class **extends** `type_container_base`

Holds a value of type T along with its type handle.

Constructor: `new(---)` — Initialises the internal type handle singleton for T.

Method	Return	Args	Description
<code>get_type_handle</code>	<code>type_handle_base</code>	—	Return the type handle for T.
<code>set</code>	<code>void</code>	T x	Store a new value, replacing any existing one.
<code>get</code>	T	—	Retrieve the stored value.

1.3 `type_match`

File: `src/types/type_match.svh`

Compile-time type-compatibility utilities. `type_match#(T1,T2)` provides static functions to check whether two type parameters are identical or whether one is derived from the other. The convenience macro ``check_is_derived_from` installs a const-initializer that will call `$fatal` if the constraint is violated at elaboration time.

`type_match#(T1, T2)`

class

Static type-comparison utilities for T1 and T2.

Method	Return	Args	Description
<code>test_is_match</code>	<code>bit</code>	—	True if T1 and T2 are the same type. Static.
<code>is_match</code>	<code>bit</code>	<code>int line,</code> <code>string file</code>	True if match; fatal error with location if not. Static.
<code>test_is_derived_from</code>	<code>bit</code>	—	True if T1 is derived from T2 (via <code>\$cast</code> probe). Static.
<code>is_derived_from</code>	<code>bit</code>	<code>int line,</code> <code>string file</code>	True if derived; fatal error if not. Static.
<code>fail_match</code>	<code>void</code>	<code>int line,</code> <code>string file</code>	Issue <code>\$fatal</code> reporting type mismatch. Static.
<code>fail_derived</code>	<code>void</code>	<code>int line,</code> <code>string file</code>	Issue <code>\$fatal</code> reporting derivation failure. Static.

1.4 typeid

File: src/types/typeid.svh

Runtime type-classification for common SystemVerilog scalar types. `typeid#(T)` uses `type_handle` singletons to determine at run time whether T is integral, two-state, four-state, real, or string. Classification is limited to explicitly enumerated built-in types.

`typeid#(T)`

class

Static type-classification functions for built-in SV scalar types.

Method	Return	Args	Description
<code>is_int</code>	<code>bit</code>	—	True if T is any integral type. Static.
<code>is_int_fail</code>	<code>bit</code>	—	Same as <code>is_int()</code> ; prints error and calls <code>\$finish</code> if false. Static.
<code>is_two_state</code>	<code>bit</code>	—	True if T is integral and two-state (e.g., <code>bit</code> , <code>int</code>). Static.
<code>is_two_state_fail</code>	<code>bit</code>	—	Same but terminates on failure. Static.

Method	Return	Args	Description
<code>is_four_state</code>	bit	—	True if T is logic, reg, integer, or time. Static.
<code>is_four_state_fail</code>	bit	—	Same but terminates on failure. Static.
<code>is_real</code>	bit	—	True if T is real. Static.
<code>is_real_fail</code>	bit	—	Same but terminates on failure. Static.
<code>is_string</code>	bit	—	True if T is string. Static.
<code>is_string_fail</code>	bit	—	Same but terminates on failure. Static.

Traits

2.1 Traits Classes

File: src/containers/traits.svh

The traits system provides per-type constants and static functions used by all parameterized containers and algorithms. Every traits class defines an `empty_t` typedef, an `empty` constant, and static `equal()`, `compare()`, and `sort()` functions. The hierarchy root is `void_traits`; scalar integer families derive from `base_int_traits`.

`void_traits`

`class extends void_t`

Root traits class. `empty` is null (`void_t`). All type-flag localparams default to false.

Method	Return	Args	Description
<code>equal</code>	<code>bit</code>	<code>void_t a, b</code>	Always returns 1 (void objects are equivalent). Static.
<code>compare</code>	<code>int32_t</code>	<code>void_t a, b</code>	Returns 0 if equal (always), else 1. Static.
<code>sort</code>	<code>void</code>	<code>ref void_t vec[\$]</code>	No-op sort. Static.
<code>check_trait</code>	<code>bit</code>	<code>bit trait, bit exp, int line, string file</code>	Assert <code>trait==exp</code> ; call <code>fail()</code> if not. Static.
<code>fail</code>	<code>void</code>	<code>int line, string file</code>	Issue \$fatal with optional file/line. Static.

`object_traits`

`class extends void_traits`

Traits for object-derived classes. Delegates `equal()/compare()` to `object::compare()`.

Method	Return	Args	Description
<code>equal</code>	<code>bit</code>	<code>object a, b</code>	True if <code>a.compare(b)==0</code> . Static.
<code>compare</code>	<code>int32_t</code>	<code>object a, b</code>	Returns <code>a.compare(b)</code> . Static.

Method	Return	Args	Description
sort	void	ref object vec[\$]	No-op (objects are unordered). Static.

`class_traits#(T)`

class **extends** void_traits

Traits for arbitrary class handles. empty is null. Uses identity (==) for equality.

Method	Return	Args	Description
equal	bit	T a, T b	True if a==b (handle identity). Static.
compare	int32_t	T a, T b	Returns 0 if equal, else 1. Static.
sort	void	ref T vec[\$]	No-op. Static.

`base_int_traits#(T)`

class **extends** void_traits

Parameterized base traits for integral scalar types. empty=T'(0). Provides arithmetic compare and built-in sort.

Method	Return	Args	Description
equal	bit	T a, T b	True if a==b. Static.
compare	int32_t	T a, T b	Returns 1 if a<b, -1 if a>b, 0 if equal. Static.
sort	void	ref T vec[\$]	Sort using SystemVerilog built-in queue sort. Static.

`base_unsigned_int_traits#(T)`

class **extends** base_int_traits#(T)

Sets is_signed=0, is_unsigned=1. Convenience base for all unsigned integer traits typedefs.

`base_signed_int_traits#(T)`

class **extends** base_int_traits#(T)

Sets is_signed=1, is_unsigned=0. Convenience base for all signed integer traits typedefs.

`base_four_state_traits#(T)`

class **extends** base_int_traits#(T)

Traits base for four-state types (logic, reg, integer, time). Sets is_two_state=0, is_four_state=1.

integer_traits

class **extends** base_four_state_traits#(integer)

Traits for the SystemVerilog integer type (four-state, signed).

reg_traits / logic_traits / time_traits

class **extends** base_four_state_traits#(reg)

Traits for reg, logic, and time (all four-state, unsigned). logic_traits and time_traits are typedefs of reg_traits.

bit_vector_traits#(N)

class **extends** base_int_traits#(bit[N-1:0])

Traits for packed bit-vector types of width N. is_array=1; is_arithmetic=1.

real_base_traits#(T)

class **extends** void_traits

Traits for real/shortreal. Uses epsilon-based equality (2.2e-16). Sorted with item comparator.

Method	Return	Args	Description
equal	bit	T a, T b	True if $a-b \leq \epsilon$. Static.
compare	int32_t	T a, T b	1 if $a < b$ (and not equal), -1 if $a > b$, else 0. Static.
sort	void	ref T vec[\$]	Sort with item comparator. Static.

string_traits

class **extends** void_traits

Traits for string. empty="". Lexicographic compare and sort.

Method	Return	Args	Description
equal	bit	string a, b	True if $a==b$. Static.
compare	int32_t	string a, b	Lexicographic: 1 if $a < b$, -1 if $a > b$, 0 if equal. Static.
sort	void	ref string vec[\$]	Sort with item comparator. Static.

Containers

3.1 object — Base Object

File: src/containers/object.svh

Defines `void_t` (the empty base class, used as the "void" type throughout SVX) and `object` (adds virtual `to_str`, `compare`, and `copy` to `void_t`). All containers and tree nodes derive ultimately from `object`.

`void_t`

class

The empty class. Serves as the null base type and as the type of the "empty" element for `void_traits`.

`object`

class **extends** `void_t`

Base object providing virtual `to_str()`, `compare()`, and `copy()`.

Method	Return	Args	Description
<code>to_str</code>	<code>string</code>	—	Return a human-readable string (default: "").
<code>compare</code>	<code>int</code>	<code>object obj</code>	Compare to <code>obj</code> ; return 0 (default implementation).
<code>copy</code>	<code>void</code>	<code>object rhs</code>	Shallow-copy from <code>rhs</code> (default: no-op).

3.2 container — Container Base Classes

File: src/containers/container.svh

Abstract container base hierarchy. `container` defines `size/clear/is_empty`. `typed_container#(T,P)` adds the protected `m_empty` member initialised from `P::empty` at construction.

`container`

virtual class **extends** `void_t`

Abstract base for all containers. Defines the core `size/clear/is_empty` interface.

Method	Return	Args	Description
<code>size</code>	<code>size_t</code>	—	Return the number of elements (pure virtual).
<code>clear</code>	<code>void</code>	—	Remove all elements (pure virtual).
<code>is_empty</code>	<code>bit</code>	—	True if container holds no elements (pure virtual).

`typed_container#(T, P)`

virtual class **extends** `container`

Typed abstract base. Initialises `m_empty` from `P::empty` in the constructor.

Constructor: `new(---)` — Casts `P::empty` to `T` and stores in protected `m_empty`.

Method	Return	Args	Description
<code>size</code>	<code>size_t</code>	—	Overridable; returns 0 by default.
<code>is_empty</code>	<code>bit</code>	—	Overridable; returns 0 by default.
<code>clear</code>	<code>void</code>	—	Overridable; no-op by default.

3.3 vector

File: `src/containers/vector.svh`

Dynamic array backed by a SystemVerilog queue. Provides random read/write with automatic extension, shallow copy/clone, equality comparison, sort, and bulk append operations. The base class for queue, stack, and deque.

`vector#(T, P)`

class **extends** `typed_container#(T,P)`

Ordered, random-access dynamic array.

Method	Return	Args	Description
<code>create</code>	<code>this_t</code>	<code>vector_type</code> <code>list</code>	Static factory: create a vector pre-loaded from a queue literal.
<code>size</code>	<code>size_t</code>	—	Number of elements currently in the vector.
<code>is_empty</code>	<code>bit</code>	—	True when <code>size()==0</code> .
<code>clear</code>	<code>void</code>	—	Delete all elements; reset to empty state.
<code>extend</code>	<code>void</code>	<code>size_t sz</code>	Push <code>sz</code> empty elements, growing the vector by that amount.
<code>write</code>	<code>void</code>	<code>index_t idx,</code> <code>T t</code>	Write element at <code>idx</code> ; auto-extends if <code>idx >= size()</code> .

Method	Return	Args	Description
<code>read</code>	<code>T</code>	<code>index_t idx</code>	Read element at <code>idx</code> ; returns <code>m.empty</code> if out of range.
<code>copy</code>	<code>void</code>	<code>this_t vec</code>	Shallow copy from <code>vec</code> into this vector.
<code>clone</code>	<code>this_t</code>	—	Return a new vector that is a shallow copy of this one.
<code>compare</code>	<code>int</code>	<code>this_t v</code>	Returns 0 if equal to <code>v</code> , else non-zero.
<code>equal</code>	<code>bit</code>	<code>this_t v</code>	Element-wise equality using <code>P::equal()</code> .
<code>sort</code>	<code>void</code>	—	Sort elements using <code>P::sort()</code> .
<code>appendv</code>	<code>void</code>	<code>vector_type v</code>	Append a queue literal to this vector.
<code>appendc</code>	<code>void</code>	<code>T t</code>	Append a single element to this vector.
<code>append</code>	<code>void</code>	<code>this_t v</code>	Append all elements of vector <code>v</code> to this vector.

3.4 map

File: `src/containers/map.svh`

Class-based dynamic associative array. Allows sparse key-value storage allocated on demand and passed by reference. `singleton_map` provides a global instance accessible via `get_inst()`.

`map#(KEY, T, P)`

class `extends` `typed_container#(T,P)`

Associative array keyed by `KEY`, storing values of type `T`.

Method	Return	Args	Description
<code>get</code>	<code>T</code>	<code>KEY key</code>	Return item for key; returns <code>m.empty</code> if key not found.
<code>insert</code>	<code>bit</code>	<code>KEY key, T item</code>	Insert (key,item). Returns 1 if key is new, 0 if duplicate (value overwritten).
<code>size</code>	<code>size_t</code>	—	Number of key-value pairs.
<code>is_empty</code>	<code>bit</code>	—	True when <code>size()==0</code> .
<code>delete</code>	<code>bit</code>	<code>KEY key</code>	Remove entry for key. Returns 1 on success, 0 if not found.
<code>clear</code>	<code>void</code>	—	Remove all entries.
<code>exists</code>	<code>bit</code>	<code>KEY key</code>	True if key exists in the map.
<code>copy</code>	<code>void</code>	<code>this_t rhs</code>	Shallow copy from <code>rhs</code> into this map.
<code>clone</code>	<code>this_t</code>	—	Return a new map that is a shallow copy of this one.
<code>compare</code>	<code>int32_t</code>	<code>this_t m</code>	Returns 0 if maps are equal, else non-zero.

Method	Return	Args	Description
<code>equal</code>	<code>bit</code>	<code>this_t m</code>	True if all keys and <code>P::equal()</code> -checked values match.
<code>first</code>	<code>bit</code>	<code>ref KEY index</code>	Move to first key (smallest); returns 0 if empty. For iterator use.
<code>last</code>	<code>bit</code>	<code>ref KEY index</code>	Move to last key (largest). For iterator use.
<code>next</code>	<code>bit</code>	<code>ref KEY index</code>	Advance to next key. For iterator use.
<code>prev</code>	<code>bit</code>	<code>ref KEY index</code>	Move to previous key. For iterator use.

`singleton_map#(KEY, T, P)`

class `extends map#(KEY,T,P)`

A map of which only one instance exists per type parameterization.

Method	Return	Args	Description
<code>get_inst</code>	<code>this_t</code>	—	Return the singleton instance (lazily created). Static.

3.5 queue

File: src/containers/queue.svh

FIFO queue derived from vector. `put()` inserts at the head; `get()` removes from the tail. `fixed_size_queue` adds a maximum-size constraint and a push-success status.

`queue#(T, P)`

class `extends vector#(T,P)`

FIFO queue: items enter at the front and leave from the back.

Method	Return	Args	Description
<code>put</code>	<code>void</code>	<code>T t</code>	Push item onto the head (front) of the queue.
<code>get</code>	<code>T</code>	—	Pop and return item from the tail; returns empty if queue is empty.
<code>peek</code>	<code>T</code>	—	Return the tail item without removing it.
<code>is_empty</code>	<code>bit</code>	—	True when <code>size()==0</code> .
<code>clone</code>	<code>this_t</code>	—	Return a shallow copy of this queue.

`fixed_size_queue#(T, P)`

class `extends queue#(T,P)`

Queue that rejects pushes when the maximum size is reached.

Constructor: `new(size_t n=1)` — Set maximum capacity to `n` (minimum 1).

Method	Return	Args	Description
<code>set_max_size</code>	<code>void</code>	<code>size_t n</code>	Update the capacity limit; clamps to current size if <code>n</code> is too small.
<code>get_max_size</code>	<code>size_t</code>	—	Return the current capacity limit.
<code>put</code>	<code>void</code>	<code>T t</code>	Push item; silently rejected if at capacity.
<code>last_push_succeeded</code>	<code>bit</code>	—	True if the most recent <code>put()</code> succeeded.
<code>is_full</code>	<code>bit</code>	—	True when <code>size() >= max_size</code> .
<code>copy</code>	<code>void</code>	<code>this_t q</code>	Copy contents and state (<code>max_size</code> , <code>push_ok</code>) from <code>q</code> .
<code>clone</code>	<code>this_t</code>	—	Return a shallow copy.
<code>equal</code>	<code>bit</code>	<code>vector#() v</code>	True if element contents, <code>max_size</code> , and <code>push_ok</code> all match.

3.6 stack

File: `src/containers/stack.svh`

LIFO stack derived from `vector`. `push()` and `pop()` operate on the front (index 0).

`stack#(T, P)`

class `extends` `vector#(T,P)`

LIFO stack: top of stack is index 0.

Method	Return	Args	Description
<code>push</code>	<code>void</code>	<code>T t</code>	Push item onto the top of the stack.
<code>pop</code>	<code>T</code>	—	Remove and return the top item; returns empty if stack is empty.
<code>peek</code>	<code>T</code>	—	Return top item without removing it.
<code>is_empty</code>	<code>bit</code>	—	True when <code>size()==0</code> .
<code>clone</code>	<code>this_t</code>	—	Return a shallow copy of this stack.

3.7 deque

File: `src/containers/deque.svh`

Double-ended queue derived from `vector`. Supports `push/pop` at both ends, `shuffle`, and `reverse`.

`deque#(T, P)`

class **extends** `vector#(T,P)`

Double-ended queue with push/pop at both front and back.

Method	Return	Args	Description
<code>push_front</code>	<code>void</code>	<code>T t</code>	Insert item at the front.
<code>push_back</code>	<code>void</code>	<code>T t</code>	Insert item at the back.
<code>pop_front</code>	<code>T</code>	—	Remove and return front item.
<code>pop_back</code>	<code>T</code>	—	Remove and return back item.
<code>shuffle</code>	<code>void</code>	—	Randomly reorder elements.
<code>reverse</code>	<code>void</code>	—	Reverse the order of elements.
<code>clone</code>	<code>this_t</code>	—	Return a shallow copy of this deque.

3.8 `circ_buf` — Circular Buffer

File: `src/containers/circ_buf.svh`

Fixed-capacity circular (ring) buffer backed by a static array of size `S`. Operates in reject mode (default) or overwrite mode. Parameterised on element type `T`, traits `P`, and compile-time size `S`.

`circ_buf#(T, P, S)`

class **extends** `typed_container#(T,P)`

Fixed-size circular buffer. `S` elements, FIFO ordering.

Constructor: `new(---)` — Calls `clear()` to initialise head, tail, and count.

Method	Return	Args	Description
<code>size</code>	<code>size_t</code>	—	Number of items currently in the buffer.
<code>is_empty</code>	<code>bit</code>	—	True when <code>count==0</code> .
<code>is_full</code>	<code>bit</code>	—	True when <code>count==S</code> .
<code>clear</code>	<code>void</code>	—	Reset head, tail, and count to zero.
<code>push</code>	<code>void</code>	<code>T t</code> , <code>circ_buf_mode_e</code> <code>mode</code>	Insert <code>t</code> ; in <code>REJECT_MODE</code> prints error if full. In <code>OVERWRITE_MODE</code> the tail item is overwritten.
<code>pop</code>	<code>T</code>	—	Remove and return head item; prints error and returns empty if buffer is empty.

3.9 `pair` / `triple` / `quadruple`

File: `src/containers/pair.svh`

Lightweight tuple containers for binding two, three, or four values of potentially different types. All extend `pair#(T1,T2)` and derive from `void_t` (use `void_traits` when placing in a container).

`pair#(T1, T2)`

class `extends` `void_t`

Holds two values of types T1 and T2.

Constructor: `new(T1 a, T2 b)` — Initialise both fields.

Method	Return	Args	Description
<code>first</code>	T1	—	Return the first element.
<code>second</code>	T2	—	Return the second element.
<code>set_first</code>	void	T1 <code>t</code>	Replace the first element.
<code>set_second</code>	void	T2 <code>t</code>	Replace the second element.

`triple#(T1, T2, T3)`

class `extends` `pair#(T1,T2)`

Extends pair with a third element of type T3.

Constructor: `new(T1 a, T2 b, T3 c)` — Initialise all three fields.

Method	Return	Args	Description
<code>third</code>	T3	—	Return the third element.
<code>set_third</code>	void	T3 <code>t</code>	Replace the third element.

`quadruple#(T1, T2, T3, T4)`

class `extends` `triple#(T1,T2,T3)`

Extends triple with a fourth element of type T4.

Constructor: `new(T1 a, T2 b, T3 c, T4 d)` — Initialise all four fields.

Method	Return	Args	Description
<code>fourth</code>	T4	—	Return the fourth element.
<code>set_fourth</code>	void	T4 <code>t</code>	Replace the fourth element.

3.10 sorter

File: `src/containers/sorter.svh`

Generic quicksort implementation for SystemVerilog queues. Based on the algorithm in *Algorithms in C* by Sedgewick. The comparator is provided by the traits parameter P.

`sorter#(T, P)`

class

Quicksort for queues of type T using P::compare() as the comparator.

Method	Return	Args	Description
<code>sort</code>	<code>void</code>	<code>ref T vec[\$]</code>	Public entry point: sort vec in place. Static.
<code>qsort</code>	<code>void</code>	<code>ref T vec[\$],</code> <code>int32_t l,</code> <code>int32_t r</code>	Recursive partition-and-sort step. Static.

Iterators

4.1 Iterator Interfaces

File: *src/iterators/iterator_intf.svh*

The iterator interface hierarchy. `iterator_intf_base` defines `set/get/skip/size/is_empty`. `fwd_intf` adds forward traversal (`first/next/is_last/at_end`). `bkwd_intf` adds backward traversal (`last/prev/is_first/at_beginning`). `random_intf` adds random selection.

`iterator_intf_base#(T, P)`

interface class

Root iterator interface. All iterators implement this.

Method	Return	Args	Description
<code>set</code>	<code>void</code>	<code>T t</code>	Write value T at the current position (pure virtual).
<code>get</code>	<code>T</code>	—	Read the value at the current position (pure virtual).
<code>skip</code>	<code>bit</code>	<code>signed_index_t d</code>	Move forward (<code>d > 0</code>) or backward (<code>d < 0</code>) by d steps (pure virtual).
<code>size</code>	<code>size_t</code>	—	Size of the bound container (pure virtual).
<code>is_empty</code>	<code>bit</code>	—	True if the iterator is unbound or the container is empty (pure virtual).

`fwd_intf#(T, P)`

interface class `extends` `iterator_intf_base#(T,P)`

Forward-traversal interface.

Method	Return	Args	Description
<code>first</code>	<code>bit</code>	—	Reset to first element; returns 0 if empty (pure virtual).
<code>next</code>	<code>bit</code>	—	Advance one step; may enter end state (pure virtual).
<code>is_last</code>	<code>bit</code>	—	True if pointing at the last element (pure virtual).

Method	Return	Args	Description
<code>at_end</code>	<code>bit</code>	—	True if past the last element (pure virtual).

`bkwd_intf#(T, P)`

interface class **extends** `iterator_intf_base#(T,P)`

Backward-traversal interface.

Method	Return	Args	Description
<code>last</code>	<code>bit</code>	—	Reset to last element; returns 0 if empty (pure virtual).
<code>prev</code>	<code>bit</code>	—	Move one step backward (pure virtual).
<code>is_first</code>	<code>bit</code>	—	True if pointing at the first element (pure virtual).
<code>at_beginning</code>	<code>bit</code>	—	True if before the first element (pure virtual).

`random_intf#(T, P)`

interface class **extends** `iterator_intf_base#(T,P)`

Random-access iterator interface.

Method	Return	Args	Description
<code>random</code>	<code>bit</code>	—	Select a random valid element (pure virtual).
<code>set_seed</code>	<code>void</code>	<code>int seed</code>	Set the RNG seed (pure virtual).
<code>set_default_seed</code>	<code>void</code>	—	Restore the default RNG seed (pure virtual).

4.2 List Iterators

File: `src/iterators/list_iterators.suh`

Iterators for vector and its subclasses (stack, queue, deque). `list_iterator_base` provides the common binding and set/get/skip/size/is_empty logic. `list_iterator` implements both forward and backward traversal. `list_random_iterator` implements random selection.

`list_iterator_base#(T, P)`

virtual class

Abstract base for list iterators. Manages binding and common index operations.

Constructor: `new(list_t list=null)` — Optionally bind a vector.

Method	Return	Args	Description
<code>bind_list</code>	<code>void</code>	<code>list_t list</code>	Bind iterator to a vector (or unbind if null).
<code>set</code>	<code>void</code>	<code>T t</code>	Write T at the current index position.
<code>get</code>	<code>T</code>	—	Read value at the current index position.
<code>skip</code>	<code>bit</code>	<code>signed_index_t d</code>	Move index by d; clamps to valid range.
<code>size</code>	<code>size_t</code>	—	Return size of the bound vector.
<code>is_empty</code>	<code>bit</code>	—	True if unbound or vector is empty.

`list_iterator#(T, P)`

class **extends** `list_iterator_base#(T,P)` **implements** `fwd_intf#(T,P)`, `bkwd_intf#(T,P)`

Bidirectional iterator over any vector-derived container.

Constructor: `new(list_t list=null)` — Optionally bind a vector.

Method	Return	Args	Description
<code>first</code>	<code>bit</code>	—	Set index to 0; returns 0 if empty.
<code>next</code>	<code>bit</code>	—	Increment index; returns 0 if past end.
<code>is_last</code>	<code>bit</code>	—	True if index is at the last valid element.
<code>at_end</code>	<code>bit</code>	—	True if index $\geq$ size.
<code>last</code>	<code>bit</code>	—	Set index to last element; returns 0 if empty.
<code>prev</code>	<code>bit</code>	—	Decrement index; returns 0 if already at start.
<code>is_first</code>	<code>bit</code>	—	True if index == 0.
<code>at_beginning</code>	<code>bit</code>	—	True if index $\leq$ 0 (before first element).

`list_random_iterator#(T, P)`

class **extends** `list_iterator_base#(T,P)` **implements** `random_intf#(T,P)`

Randomly selects an element from a bound vector.

Constructor: `new(list_t list=null)` — Optionally bind a vector; sets default RNG seed.

Method	Return	Args	Description
<code>random</code>	<code>bit</code>	—	Uniformly choose a random index in [0, size-1].
<code>set_seed</code>	<code>void</code>	<code>int seed</code>	Seed the RNG via <code>\$urandom(seed)</code> .
<code>set_default_seed</code>	<code>void</code>	—	Reset to the default seed (1).

4.3 Map Iterators

File: *src/iterators/map_iterators.svh*

Iterators for the map container. `map_iterator_base` handles binding, index access, size, and `is_empty`. `map_iterator` provides bidirectional traversal. `map_random_iterator` provides random selection. Map keys are traversed in the natural order of the underlying SV associative array.

`map_iterator_base#(KEY, T, P)`

virtual class

Abstract base for map iterators. Stores map binding and current key index.

Constructor: `new(map_t map=null)` — Optionally bind a map.

Method	Return	Args	Description
<code>bind_map</code>	<code>void</code>	<code>map_t m</code>	Bind iterator to map m.
<code>get_index</code>	<code>KEY</code>	—	Return the current key.
<code>size</code>	<code>size_t</code>	—	Return number of entries in the bound map.
<code>is_empty</code>	<code>bit</code>	—	True if unbound or map is empty.

`map_iterator#(KEY, T, P)`

class **extends** `map_iterator_base#(KEY,T,P)` **implements** `fwd_intf#(T,P)`, `bkwd_intf#(T,P)`

Bidirectional iterator over a map. Tracks a four-state (INVALID/FIRST/VALID/LAST) traversal state.

Constructor: `new(map_t map=null)` — Optionally bind a map; set state to INVALID.

Method	Return	Args	Description
<code>set</code>	<code>void</code>	<code>T t</code>	Update the value at the current key.
<code>get</code>	<code>T</code>	—	Return the value at the current key.
<code>get_index</code>	<code>KEY</code>	—	Return the current key.
<code>first</code>	<code>bit</code>	—	Move to first (smallest) key; set state FIRST.
<code>next</code>	<code>bit</code>	—	Advance to next key; set state LAST if at end.
<code>is_last</code>	<code>bit</code>	—	True if current key equals the last key in the map.
<code>at_end</code>	<code>bit</code>	—	True when state==LAST.
<code>last</code>	<code>bit</code>	—	Move to last key; set state LAST.
<code>prev</code>	<code>bit</code>	—	Move to previous key; set state FIRST if at start.
<code>is_first</code>	<code>bit</code>	—	True if current key equals first key in the map.
<code>at_beginning</code>	<code>bit</code>	—	True when state==FIRST.

Method	Return	Args	Description
<code>skip</code>	<code>bit</code>	<code>signed_index_t d</code>	Skip <code>d</code> steps via repeated <code>next()/prev()</code> calls.

`map_random_iterator#(KEY, T, P)`

class `extends` `map_iterator_base#(KEY,T,P)` `implements` `random_intf#(T,P)`

Randomly selects a map entry.

Constructor: `new(map_t map=null)` — Optionally bind a map; set default RNG seed.

Method	Return	Args	Description
<code>set</code>	<code>void</code>	<code>T t</code>	Update value at current key.
<code>get</code>	<code>T</code>	—	Return value at current key.
<code>random</code>	<code>bit</code>	—	Select a uniformly random entry.
<code>set_seed</code>	<code>void</code>	<code>int seed</code>	Seed the RNG via <code>\$urandom(seed)</code> .
<code>set_default_seed</code>	<code>void</code>	—	Reset to default seed (1).

4.4 Permutation Iterators

File: `src/iterators/permute_iterators.svh`

Iterators that generate all permutations of a bound vector without modifying it. The permutation vector `pv` holds indices into the bound vector; `get_nth(n)` returns the `nth` element for the current permutation. `permute_iterator` traverses permutations sequentially; `permute_random_iterator` jumps to a randomly chosen permutation.

`permute_iterator_base#(T, P)`

class

Shared permutation state and algorithm. Stores `pv[]`, `pix`, and `max_pix`.

Constructor: `new(vec_t v=null)` — Optionally bind a vector and initialise `pv`.

Method	Return	Args	Description
<code>bind_vector</code>	<code>void</code>	<code>vec_t v</code>	Bind a vector; calls <code>initialize()</code> .
<code>size</code>	<code>size_t</code>	—	Size of the bound vector.
<code>is_empty</code>	<code>bit</code>	—	True if unbound or empty.
<code>get_nth</code>	<code>T</code>	<code>index_t n</code>	Return the <code>nth</code> element for the current permutation.
<code>get_permutation_index</code>	<code>longint</code>	—	Return the current permutation index (0-based).
<code>get_permutation_vector</code>	<code>vector#()</code>	—	Return a copy of <code>pv</code> as a <code>vector#(index_t, uint64_traits)</code> .

Method	Return	Args	Description
<code>skip</code>	<code>bit</code>	<code>signed_index_t d</code>	Advance or retreat <code>pix</code> by <code>d</code> permutations.
<code>print</code>	<code>void</code>	—	Display current <code>pix</code> and <code>pv</code> to <code>stdout</code> .

`permute_iterator#(T, P)`

class `extends permute_iterator_base#(T,P)` **implements** `fwd_intf#(T,P)`, `bkwd_intf#(T,P)`

Sequential bidirectional traversal of all permutations.

Constructor: `new(vec_t v=null)` — Optionally bind a vector.

Method	Return	Args	Description
<code>first</code>	<code>bit</code>	—	Reset to permutation 0.
<code>next</code>	<code>bit</code>	—	Advance to next permutation; returns 0 if already at last.
<code>is_last</code>	<code>bit</code>	—	True if at the last permutation.
<code>at_end</code>	<code>bit</code>	—	True if <code>pix</code> $\geq$ <code>max_pix</code> .
<code>last</code>	<code>bit</code>	—	Jump to the last permutation.
<code>prev</code>	<code>bit</code>	—	Return to the previous permutation.
<code>is_first</code>	<code>bit</code>	—	True if <code>pix==0</code> .
<code>at_beginning</code>	<code>bit</code>	—	True if <code>pix<0</code> .

`permute_random_iterator#(T, P)`

class `extends permute_iterator_base#(T,P)` **implements** `random_intf#(T,P)`

Jump to a randomly chosen permutation.

Constructor: `new(vec_t v=null)` — Optionally bind a vector; set default RNG seed.

Method	Return	Args	Description
<code>random</code>	<code>bit</code>	—	Select a uniformly random permutation index.
<code>set_seed</code>	<code>void</code>	<code>int seed</code>	Seed the RNG via <code>\$urandom(seed)</code> .
<code>set_default_seed</code>	<code>void</code>	—	Reset to default seed (1).

4.5 range

File: src/iterators/range.svh

A view over a contiguous sub-range `[lb, ub]` of an existing iterator. `range_base` stores and validates the bounds. `range#(T,P)` wraps a `fwd_intf` iterator and optionally a `bkwd_intf` if the underlying iterator supports it; backward access is silently unavailable otherwise.

range_base

virtual class

Stores validated lower/upper index bounds. Swaps lb/ub if inverted.

Constructor: `new(size_t size, index_t lb, index_t ub)` — Clamp and optionally swap the bounds.

Method	Return	Args	Description
<code>get_lower_bound</code>	<code>index_t</code>	—	Return lb.
<code>get_upper_bound</code>	<code>index_t</code>	—	Return ub.

range#(T, P)

class **extends** `range_base` **implements** `fwd_intf#(T,P)`, `bkwd_intf#(T,P)`

Sub-range iterator. Binds to a `fwd_intf`; optionally acquires backward capability via `$cast`.

Constructor: `new(iterator_intf_base#() it, index_t lb, ub)` — Cast it to `fwd_intf` (fatal if fails); `$cast` to `bkwd_intf` (null if fails).

Method	Return	Args	Description
<code>set</code>	<code>void</code>	<code>T t</code>	Set value at current iterator position.
<code>get</code>	<code>T</code>	—	Get value at current iterator position.
<code>size</code>	<code>size_t</code>	—	Number of elements in the range: <code>ub-lb+1</code> .
<code>is.empty</code>	<code>bit</code>	—	True if underlying iterator is null or empty.
<code>first</code>	<code>bit</code>	—	Move underlying iterator to position lb.
<code>next</code>	<code>bit</code>	—	Advance one step within range bounds.
<code>is.last</code>	<code>bit</code>	—	True if current position $i = ub$.
<code>at.end</code>	<code>bit</code>	—	True if position $i \geq ub$.
<code>last</code>	<code>bit</code>	—	Move to position ub (requires <code>bkwd_intf</code>).
<code>prev</code>	<code>bit</code>	—	Move one step backward (requires <code>bkwd_intf</code>).
<code>is.first</code>	<code>bit</code>	—	True if current position $= lb$.
<code>at.beginning</code>	<code>bit</code>	—	True if position $i \leq lb$.
<code>skip</code>	<code>bit</code>	<code>signed_index_t d</code>	Delegate to underlying <code>fwd_iter.skip(d)</code> .

4.6 map_range

File: src/iterators/map_range.svh

Extends `range` for map iterators, adding `get_index()` to retrieve the current map key from within a sub-range view.

`map_range#(KEY, T, P)`

class `extends range#(T,P)`

Range view over a map iterator slice, exposing the current key.

Constructor: `new(map_iter_t it, index_t lb, ub)` — Delegate to `range`; retain reference to the map iterator.

Method	Return	Args	Description
<code>get_index</code>	KEY	—	Return the key at the current iterator position.

Algorithms

5.1 algo — Core Algorithms

File: src/algorithms/algo.svh

Generic single-iterator algorithms. All functions are static and accept a `fwd_intf` iterator plus, where needed, a `predicate` or `fcn` object. `find()` positions the iterator at the first matching element rather than returning it.

`algo#(T, P)`

class

Collection of generic single-iterator algorithms.

Method	Return	Args	Description
<code>count</code>	<code>uint32_t</code>	<code>fwd_intf#()</code> <code>iter,</code> <code>predicate#(T)</code> <code>p</code>	Count elements for which <code>p.is_true()</code> . Static.
<code>all_of</code>	<code>bit</code>	<code>fwd_intf#()</code> <code>iter,</code> <code>predicate#(T)</code> <code>p</code>	True if predicate holds for every element. Static.
<code>none_of</code>	<code>bit</code>	<code>fwd_intf#()</code> <code>iter,</code> <code>predicate#(T)</code> <code>p</code>	True if predicate holds for no element. Static.
<code>any_of</code>	<code>bit</code>	<code>fwd_intf#()</code> <code>iter,</code> <code>predicate#(T)</code> <code>p</code>	True if predicate holds for at least one element. Static.
<code>min</code>	<code>T</code>	<code>fwd_intf#()</code> <code>iter</code>	Return minimum element using <code>P::compare()</code> . Static.
<code>max</code>	<code>T</code>	<code>fwd_intf#()</code> <code>iter</code>	Return maximum element using <code>P::compare()</code> . Static.
<code>find</code>	<code>void</code>	<code>fwd_intf#()</code> <code>iter,</code> <code>predicate#(T)</code> <code>p</code>	Position <code>iter</code> at first element where <code>p.is_true()</code> ; no-op if not found. Static.

Method	Return	Args	Description
<code>for_each</code>	<code>void</code>	<code>fwd_intf#()</code> <code>iter, fcn#(T)</code> <code>fn</code>	Call <code>fn.f(element)</code> for every element. Static.

5.2 algo2 — Dual-Iterator Algorithms

File: *src/algorithms/algo2.svh*

Algorithms that operate on two iterators simultaneously. `zip` traverses both iterators in lock-step, calling `fn.f(t1, t2)` at each step until either iterator reaches its end.

`algo2#(T1, P1, T2, P2)`

class

Algorithms for paired traversal of two differently-typed iterators.

Method	Return	Args	Description
<code>zip</code>	<code>void</code>	<code>fwd_intf#(T1,P1)</code> Traverse iter1 and iter2 in lock-step; <code>iter1,</code> call <code>fn.f(t1,t2)</code> at each step. Stops <code>fwd_intf#(T2,P2)</code> when either iterator ends. Static. <code>iter2,</code> <code>fcn2#(T1,T2)</code> <code>fn</code>	

5.3 accum — Accumulation Algorithms

File: *src/algorithms/accum.svh*

Algorithms that carry a mutable accumulator `A` across iterations. `accum` operates on one iterator; `accum2` operates on two in lock-step.

`accum#(T, P, A)`

class

Single-iterator accumulation algorithm.

Method	Return	Args	Description
<code>accumulate</code>	<code>void</code>	<code>fwd_intf#()</code> <code>iter,</code> <code>accum_fcn#(T,A)</code> <code>fn, ref A acc</code>	Call <code>fn.f(element, acc)</code> for every element; <code>acc</code> is updated in place by <code>fn</code> . Static.

`accum2#(T1, P1, T2, P2, A)`

class

Dual-iterator accumulation algorithm.

Method	Return	Args	Description
<code>accumulate</code>	<code>void</code>	<code>fwd_intf#(T1,P1)</code> <code>iter1,</code> <code>fwd_intf#(T2,P2)</code> <code>iter2,</code> <code>accum_fcn2#()</code> <code>fn, ref A acc</code>	Traverse both iterators in lock-step; call <code>fn.f(t1, t2, acc)</code> at each step. Static.

5.4 fcn — Function Objects

File: src/algorithms/fcn.svh

Virtual function objects (SVX's substitute for C++ functors and lambdas). Users derive concrete classes and override `f()`. The hierarchy provides one-argument, two-argument, and accumulating variants.

`fcn_base`

virtual class

Empty base class for all function objects.

`fcn#(T)`

virtual class **extends** `fcn_base`

Abstract function object taking one argument of type `T`.

Method	Return	Args	Description
<code>f</code>	<code>void</code>	<code>T t</code>	Apply the function to <code>t</code> (pure virtual).

`fcn2#(T1, T2)`

virtual class **extends** `fcn_base`

Abstract function object taking two arguments of types `T1` and `T2`.

Method	Return	Args	Description
<code>f</code>	<code>void</code>	<code>T1 t1, T2 t2</code>	Apply the function to <code>(t1, t2)</code> (pure virtual).

`accum_fcn#(T, A)`

virtual class `extends fcn_base`

Abstract function object taking one value and a mutable accumulator.

Method	Return	Args	Description
<code>f</code>	<code>void</code>	<code>T t, ref A a</code>	Apply the function; update accumulator <code>a</code> in place (pure virtual).

`accum_fcn2#(T1, T2, A)`

virtual class `extends fcn_base`

Abstract function object taking two values and a mutable accumulator.

Method	Return	Args	Description
<code>f</code>	<code>void</code>	<code>T1 t1, T2 t2, ref A a</code>	Apply the function to <code>(t1, t2)</code> ; update <code>a</code> in place (pure virtual).

5.5 predicate — Predicate Classes

File: `src/algorithms/predicate.svh`

Virtual predicate hierarchy. A predicate is a Boolean function over a single element `T`. Concrete predicate classes are composed with `and_pred`, `or_pred`, and `not_pred`.

`predicate_base`

virtual class

Empty root for all predicate classes.

`predicate#(T)`

virtual class `extends predicate_base`

Abstract predicate: `is_true()` is pure virtual; `is_false()` delegates.

Method	Return	Args	Description
<code>is_true</code>	<code>bit</code>	<code>T t</code>	Return true if <code>t</code> satisfies the predicate (pure virtual).
<code>is_false</code>	<code>bit</code>	<code>T t</code>	Return <code>!is_true(t)</code> .

`and_pred#(T)`

class `extends predicate#(T)`

True if both sub-predicates `a` and `b` are true.

Constructor: `new(predicate#(T) _a, _b)` — Store references to a and b.

Method	Return	Args	Description
<code>is_true</code>	<code>bit</code>	<code>T t</code>	Return <code>a.is_true(t) && b.is_true(t)</code> .

`or_pred#(T)`

class `extends` `predicate#(T)`

True if at least one of the two sub-predicates is true.

Constructor: `new(predicate#(T) _a, _b)` — Store references to a and b.

Method	Return	Args	Description
<code>is_true</code>	<code>bit</code>	<code>T t</code>	Return <code>a.is_true(t) b.is_true(t)</code> .

`not_pred#(T)`

class `extends` `predicate#(T)`

True when the wrapped predicate is false.

Constructor: `new(predicate#(T) _p)` — Store reference to p.

Method	Return	Args	Description
<code>is_true</code>	<code>bit</code>	<code>T t</code>	Return <code>p.is_false(t)</code> .

`always_true#(T)`

class `extends` `predicate#(T)`

Constant predicate that always evaluates to 1.

Method	Return	Args	Description
<code>is_true</code>	<code>bit</code>	<code>T t</code>	Always returns 1.

`always_false#(T)`

class `extends` `predicate#(T)`

Constant predicate that always evaluates to 0.

Method	Return	Args	Description
<code>is_true</code>	<code>bit</code>	<code>T t</code>	Always returns 0.

Linked Structures

6.1 node

File: src/linked/node.svh

A named, markable container node extending `object`. The name field supports hierarchical naming in the tree. The mark/unmark mechanism supports traversal algorithms that need to track visited nodes.

`node`

class **extends** `object`

Named, markable base node for tree structures.

Constructor: `new(string nm="")` — Initialise name to nm.

Method	Return	Args	Description
<code>set_name</code>	<code>void</code>	<code>string nm</code>	Set the node name.
<code>get_name</code>	<code>string</code>	—	Return the node name.
<code>mark</code>	<code>void</code>	—	Set the mark flag to 1.
<code>unmark</code>	<code>void</code>	—	Clear the mark flag to 0.
<code>get_mark</code>	<code>bit</code>	—	Return the current mark value.
<code>is_marked</code>	—	—	Return the mark flag (equivalent to <code>get_mark()</code>).

6.2 tree

File: src/linked/tree.svh

Classic tree data structure. Each node has a name, a reference to its parent, and a map of named children. Hierarchical path names use the dot (.) separator. The `find()` method parses a dot-separated path using the lexer and locates the named node.

`tree`

class **extends** `node`

N-ary tree node with named children, full-path support, and hierarchical search.

Constructor: `new(string nm, tree p)` — Set name, link to parent p; register as child of p; build `full_name`.

Method	Return	Args	Description
<code>insert</code>	<code>void</code>	<code>tree t</code>	Add <code>t</code> as a named child of this node; error if name duplicate.
<code>get_parent</code>	<code>tree</code>	—	Return the parent node handle.
<code>get_full_name</code>	<code>string</code>	—	Return the fully-qualified dotted path name.
<code>first_child</code>	<code>bit</code>	<code>ref string nm</code>	Set <code>nm</code> to the name of the first child; returns 0 if none.
<code>next_child</code>	<code>bit</code>	<code>ref string nm</code>	Advance <code>nm</code> to the next sibling name; returns 0 if none.
<code>num_children</code>	<code>size_t</code>	—	Return the number of direct children.
<code>size</code>	<code>size_t</code>	—	Return the number of direct children (delegates to <code>get_children()</code>).
<code>get_children</code>	<code>deque#()</code>	—	Return a deque of all direct child handles.
<code>get_child</code>	<code>tree</code>	<code>string nm</code>	Return the named child; null if not found.
<code>find</code>	<code>tree</code>	<code>string path</code>	Locate a node by dot-separated path. Returns null on parse error or not found.
<code>mark_all</code>	<code>void</code>	—	Recursively mark all nodes in this subtree.
<code>unmark_all</code>	<code>void</code>	—	Recursively unmark all nodes in this subtree.

6.3 tree_iterator

File: *src/linked/tree_iterator.svh*

Pre-order depth-first iterator over a tree. A stack drives the traversal: `first()` seeds the stack with the root; `next()` pops the current node and pushes its children. `size()` counts all reachable nodes using a separate DFS with marks.

`tree_iterator#(T, P)`

class `implements fwd_intf#(T,P)`

Forward-only pre-order DFS iterator over a tree.

Constructor: `new(tree t=null)` — Bind to tree `t`; initialise internal stack.

Method	Return	Args	Description
<code>size</code>	<code>size_t</code>	—	Count all nodes with a separate DFS using marks. Side effect: alters node marks.
<code>is_empty</code>	<code>bit</code>	—	True if <code>m_tree</code> is null.

Method	Return	Args	Description
<code>set</code>	<code>void</code>	<code>tree t</code>	No-op (required by <code>fwd_intf</code> ; not applicable to tree traversal).
<code>get</code>	<code>tree</code>	—	Return the tree node at the top of the stack.
<code>first</code>	<code>bit</code>	—	Reset stack; push root; unmark all nodes.
<code>next</code>	<code>bit</code>	—	Pop current node; push all its children onto the stack.
<code>is_last</code>	<code>bit</code>	—	True when exactly one node remains on the stack.
<code>at_end</code>	<code>bit</code>	—	True when the stack is empty.
<code>skip</code>	<code>bit</code>	<code>signed_index_t d</code>	Skip <code>d</code> nodes forward (negative <code>d</code> not supported).

Set

7.1 set

File: src/set/set_impl.svh

Mathematical set backed by a `map#(T, uint32_t, uint32_traits)`. Because map keys are maintained in sorted order by the SV associative array, set operations such as intersection use a sorted merge-walk ($O(n+m)$) rather than membership tests. All set operations return a new set and do not mutate their operands.

`set#(T, P)`

class **extends** `typed_container#(T,P)`

Mathematical set with insert, membership test, and set-algebra operations.

Constructor: `new(---)` — Allocate the internal map.

Method	Return	Args	Description
<code>size</code>	<code>size_t</code>	—	Return the number of members.
<code>is_empty</code>	<code>bit</code>	—	True when the set has no members.
<code>clear</code>	<code>void</code>	—	Remove all members.
<code>insert</code>	<code>void</code>	<code>T item</code>	Add item to the set (duplicates ignored via map semantics).
<code>contains</code>	<code>bit</code>	<code>T item</code>	True if item is a member.
<code>intersection</code>	<code>set_t</code>	<code>set_t s</code>	Return a new set of elements in both this and s. Uses sorted merge-walk.
<code>set_union</code>	<code>set_t</code>	<code>set_t s</code>	Return a new set of all elements in this or s (or both).
<code>difference</code>	<code>set_t</code>	<code>set_t s</code>	Return a new set of elements in this but not in s.
<code>to_vector</code>	<code>vector#()</code>	—	Return a vector containing all set members in key order.
<code>from_vector</code>	<code>void</code>	<code>vector#() vec</code>	Insert all elements of vec into the set (skipping duplicates).